

Министерство образования и науки Российской Федерации
Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет
ИТМО»

ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ
Направление подготовки: 11.03.02 Инфокоммуникационные технологии
и системы связи

Лабораторная работа №7

Выполнил:
Бацанова Екатерина Александровна
Группа К3323
Проверил:
Марченко Елена Вадимовна

Санкт-Петербург
2025

СОДЕРЖАНИЕ

Стр.

Цель	3
Ход работы	4
Вывод.....	21

ЦЕЛЬ

Цель лабораторной работы – разработать простой веб-проект на Django с нуля, реализующий структуру из трёх страниц (главная, о разработке, обратная связь), с использованием шаблонов и форм, демонстрируя базовые возможности Django.

Для выполнения работы можно выделить следующие цели:

1. Создать Django-проект и базовое приложение; Настроить маршруты (URLconf);
2. Создать шаблоны (templates);
3. Создать три страницы;
4. Реализовать форму обратной связи;
5. Организовать хранение и стили.

ХОД РАБОТЫ

Для начала работы установим Django с помощью команды

```
pip install django
```

```
(.venv) aleksandrbacanov@MacBook-Pro-Aleksandr-3 lab_7-8 % pip install django
Collecting django
  Obtaining dependency information for django from https://files.pythonhosted.org/packages/63/e0/6a5b5ea350c5bd63fe94b05e4c146c18facb51229d9dee42aa39f9fc2214/Django-5.2-py3-none-any.whl.metadata
  Using cached Django-5.2-py3-none-any.whl.metadata (4.1 kB)
Collecting asgiref<=3.8.1 (from django)
  Obtaining dependency information for asgiref<=3.8.1 from https://files.pythonhosted.org/packages/39/e3/893e8757be2612e6c266d9bb58ad2e3651524b5b40cf56761e985a28b13e/asgiref-3.8.1-py3-none-any.whl.metadata
  Using cached asgiref-3.8.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse<=0.3.1 (from django)
```

Рисунок 1 — Установка Django.

Затем создадим новый Django-проект с базовой структурой с помощью:

```
django-admin startproject mysite
```

После этого у нас появляется следующая структура проекта:

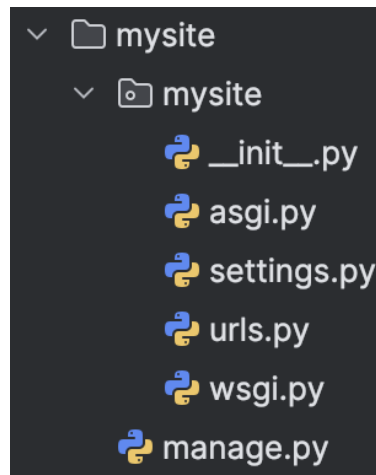


Рисунок 2 — Структура каталога mysite.

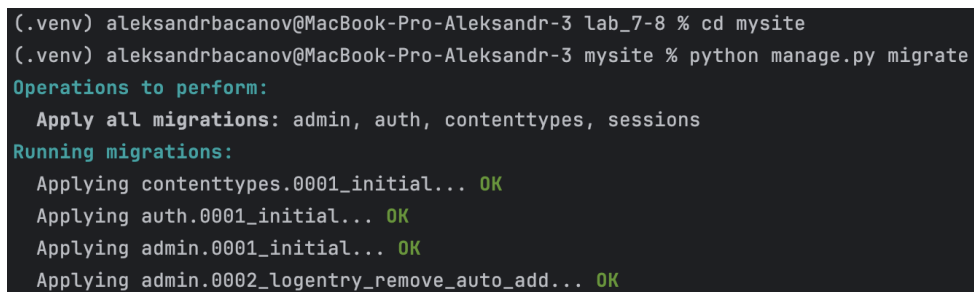
Каждый файл выполняет свою функцию:

- `manage.py`
это инструмент командной строки для управления проектом, с его помощью можно запускать сервер, создавать миграции, запускать тесты и тд.;

- `__init__.py`
делает папку python-модулем, нужен, чтобы python мог импортировать файлы отсюда;
- `settings.py`
подключает базы данных, указывает установленные приложения (`INSTALLED_APPS`), пути к шаблонам, статике и медиа, настройки безопасности и локализации;
- `urls.py`
определяет, какие URL-адреса к каким представлениям (`views`) ведут, обычно делегирует маршруты в другие приложения через `include()`.
- `wsgi.py`
интерфейс между Django и WSGI-сервером (например, Gunicorn, uWSGI), используется для запуска проекта в продакшене (обычно для синхронных запросов);
- `asgi.py`
то же самое, но для асинхронного сервера (например, Daphne, Uvicorn), нужен, если используем WebSocket или асинхронные задачи.

Перейдем в директорию Django-проекта, применим миграции (то есть создадим или обновим таблицы в базе данных, согласно моделям Django), и попробуем запустить приложение с помощью:

```
python manage.py runserver
```



```
(.env) aleksandrbacanov@MacBook-Pro-Aleksandr-3 lab_7-8 % cd mysite
(.env) aleksandrbacanov@MacBook-Pro-Aleksandr-3 mysite % python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
```

Рисунок 3 — Первоначальное применение миграций.

После этого перейдя по адресу <http://127.0.0.1:8000/> мы увидим базовую страницу django – значит, мы все сделали верно.

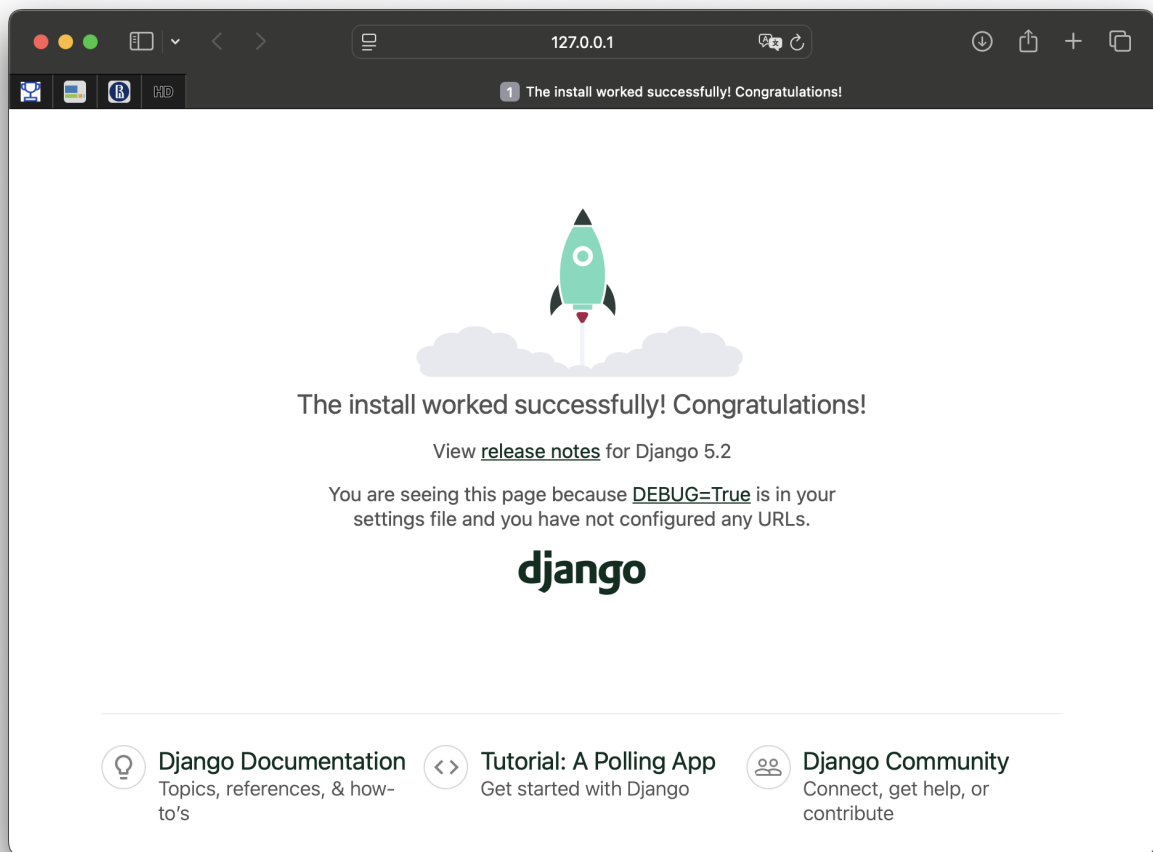


Рисунок 4 — Базовая страница django.

Теперь создадим новое приложение Django с именем main:

```
python manage.py startapp main
```

После этого появится новая директория, имеющая следующую структуру (рис. 5)

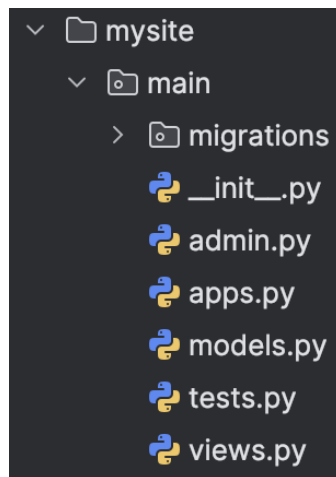


Рисунок 5 — Структура каталога main

Каждый файл также выполняет свою функцию:

- `admin.py`
позволяет зарегистрировать модели, чтобы они отображались в административной панели Django, а также настроить, как они будут отображаться и редактироваться;
- `apps.py`
в этом файле Django хранит настройки для приложения, такие как его название, метаданные и настройки;
- `migrations`
папка для хранения миграций базы данных, как уже было сказано: миграции – это файлы, которые описывают изменения в базе данных (например, создание или изменение таблиц), основанные на моделях;
- `models.py`
здесь определяются модели (структура базы данных), создаются классы, которые Django будет использовать для создания таблиц в базе данных;
- `tests.py`
в этом файле можно определить автоматические тесты, чтобы убедиться, что всё работает корректно;

- `views.py`

отвечает за определение представлений (`views`), которые отвечают за логику обработки запросов и возврат ответа пользователю, здесь определяются функции или классы, которые обрабатывают запросы и возвращают нужные страницы или данные.

В `INSTALLED_APPS` добавим наше приложение `main`.

```
33     INSTALLED_APPS = [  
34         "django.contrib.admin",  
35         "django.contrib.auth",  
36         "django.contrib.contenttypes",  
37         "django.contrib.sessions",  
38         "django.contrib.messages",  
39         "django.contrib.staticfiles",  
40         "main",  
41     ]
```

Рисунок 6 — Изменения в `mysite/settings.py`.

Создадим структуру шаблонов:

```
mkdir -p main/templates/main  
mkdir -p main/static/main
```

Теперь создаем базовый шаблон – он будет определять `header` и `footer`.


```

1  {% load static %}
2  <!DOCTYPE html>
3  <html lang="ru">
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>{% block title %}Мой Django Сайт{% endblock %}</title>
8      <link rel="stylesheet" href="{% static 'main/css/style.css' %}">
9  </head>
10 <body>
11 <header class="header" role="banner">
12     <div class="container header-inner">
13         <h1 class="logo" aria-label="Логотип сайта">Мой Django Сайт</h1>
14         <nav class="nav" role="navigation" aria-label="Основная навигация">
15             <a href="{% url 'home' %}">Главная</a>
16             <a href="{% url 'about' %}">О разработке</a>
17             <a href="{% url 'contact' %}">Обратная связь</a>
18         </nav>
19     </div>
20 </header>
21 <main class="main-content container" role="main">
22     {% block content %}
23     {% endblock %}
24 </main>
25 <footer class="footer" role="contentinfo">
26     <div class="container footer-inner">
27         
28         <p>© 2025 Мой сайт. Все права защищены.</p>
29     </div>
30 </footer>
31 <script>
32 document.addEventListener("DOMContentLoaded", () => {
33     const btn = document.getElementById('heartBtn');
34
35     btn.addEventListener('click', (e) => {
36         const heart = document.createElement('div');
37         heart.classList.add('flying-heart');
38         heart.innerText = '❤️';
39
40         const rect = btn.getBoundingClientRect();
41         const offsetX = (Math.random() - 0.5) * 60; // разброс по X
42         const offsetY = (Math.random() - 1) * 40; // вверх и чуть в стороны
43
44         heart.style.left = `${rect.left + rect.width / 2 + offsetX}px`;
45         heart.style.top = `${window.scrollY + rect.top + offsetY}px`;
46
47         document.body.appendChild(heart);
48
49         setTimeout(() => heart.remove(), 1200);
50     });
51 });
52 </script>
53 </body>
54 </html>

```

Рисунок 7 — Код файла *main/templates/main/base.html*.

Внесем изменения в *main/views.py* – добавим три представления для рендеринга страниц: для главной страницы (*home_view*), страницы о разработке (*about_view*) и страницы обратной связи (*contact_view*). На странице обратной связи реализована обработка формы с помощью *ContactForm*: при отправке формы (метод POST) данные проверяются на валидность, и если форма правильная, выводится сообщение об успешной отправке. После этого форма очищается, готовая для следующего ввода. Также используется система сообщений Django для уведомления пользователя о статусе отправки.

```
1  from django.shortcuts import render
2  from .forms import ContactForm
3  from django.contrib import messages
4  from .models import ContactMessage
5
6  def home_view(request):
7      return render(request, template_name: 'main/home.html')
8
9  def about_view(request):
10     return render(request, template_name: 'main/about.html')
11
12  def contact_view(request):
13     if request.method == 'POST':
14         form = ContactForm(request.POST)
15         if form.is_valid():
16             # Сохраняем данные в базу
17             ContactMessage.objects.create(
18                 name=form.cleaned_data['name'],
19                 email=form.cleaned_data['email'],
20                 subject=form.cleaned_data['subject'],
21                 message=form.cleaned_data['message']
22             )
23             messages.success(request, message: 'Сообщение отправлено!')
24             form = ContactForm() # очистить форму
25         else:
26             # Вывод ошибок формы, если они есть
27             messages.error(request, message: 'Произошла ошибка при отправке сообщения.')
28     else:
29         form = ContactForm()
30     return render(request, template_name: 'main/contact.html', context: {'form': form})
```

Рисунок 8 — Код файла *main/views.py*.

Внесем изменения в *main/urls.py* – создадим маршруты для обработки URL-адресов в проекте. Для главной страницы (""), страницы о разработке

('about/') и страницы обратной связи ('contact/') определены соответствующие пути, которые связываются с функциями представлений.

```
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path('', views.home_view, name='home'),
6     path('about/', views.about_view, name='about'),
7     path('contact/', views.contact_view, name='contact'),
8 ]
```

Рисунок 9 — Код файла *main/urls.py*.

Подключим *main/urls.py* в *mysite/urls.py*

```
18 from django.contrib import admin
19 from django.urls import path, include
20
21 urlpatterns = [
22     path("admin/", admin.site.urls),
23     path('', include('main.urls')),
24 ]
```

Рисунок 10 — Код файла *mysite/urls.py*.

Создадим форму обратной связи в *main/forms.py*

```
1 from django import forms
2
3 class ContactForm(forms.Form): 4 usages
4     name = forms.CharField(label='Ваше имя', max_length=100)
5     email = forms.EmailField(label='Ваш email')
6     subject = forms.CharField(label='Тема', max_length=200)
7     message = forms.CharField(label='Сообщение', widget=forms.Textarea(attrs={'rows': 6, 'cols': 40}))
```

Рисунок 11 — Код файла *main/forms.py*.

Теперь создадим все необходимые страницы. Начнем с главной — *home.html*

```

1  {% extends 'main/base.html' %}
2  {% block title %}Главная{% endblock %}
3  {% block content %}
4      <section class="hero">
5          <h2>Добро пожаловать на мой Django-сайт!</h2>
6          <p>Этот сайт демонстрирует возможности веб-разработки с использованием Django.
7              Здесь вы можете узнать больше о проекте.</p>
8          <a href="{% url 'about' %}" class="btn">Узнать о разработке</a>
9          <p>А здесь вы сможете связаться со мной.</p>
10         <a href="{% url 'contact' %}" class="btn">Отправить обратную связь</a>
11     </section>
12     <section class="like-section">
13         <h3>Вам нравится сайт?</h3>
14         <p>Нажмите на сердечко, если вам понравился сайт!</p>
15
16         <button id="heartBtn" class="heart-button" aria-label="Поставить лайк">
17             ❤️
18         </button>
19         <div id="heartContainer"></div>
20     </section>
21 {% endblock %}

```

Рисунок 12 — Код файла *home.html*.

Следующая страница – о разработке *about.html*

```

1  {% extends 'main/base.html' %}
2  {% load static %}
3  {% block title %}0 разработке{% endblock %}
4  {% block content %}
5      <div class="about-page">
6
7          <section class="intro" id="about">
8              <h2>0 проекте</h2>
9              <p>
10                 Этот сайт разработан с использованием фреймворка <strong>Django</strong> и
11                 демонстрирует современные подходы к веб-разработке на языке <strong>Python</strong>.
12                 Цель проекта — создать полнофункциональный, адаптивный веб-интерфейс с удобной
13                 навигацией, понятной архитектурой и возможностью масштабирования.
14             </p>
15         </section>
16         <section class="features" id="features">
17             <h3>Ключевые возможности</h3>
18             <ul>
19                 <li>Многостраничная структура с общей навигацией и шапкой</li>
20                 <li>Форма обратной связи с валидацией и сохранением данных</li>
21                 <li>Адаптивный дизайн с лёгкими анимациями</li>
22                 <li>Интеграция с Django Admin для управления содержимым</li>
23                 <li>Простая расширяемость под дополнительные модули</li>
24             </ul>
25         </section>
26         <section class="tech" id="tech">
27             <h3>Используемые технологии</h3>
28             <div class="tech-list">
29                 <div class="tech-item">
30                     
31                     <p>Python</p>
32                 </div>
33                 <div class="tech-item">
34                     
35                     <p>Django</p>
36                 </div>
37                 <div class="tech-item">
38                     
39                     <p>HTML5</p>
40                 </div>
41                 <div class="tech-item">
42                     
43                     <p>CSS3</p>
44                 </div>
45             </div>
46         </section>
47         <section class="faq" id="faq">
48             <h3>Вопросы и ответы</h3>
49             <details>
50                 <summary>Как работает форма обратной связи?</summary>
51                 <p>Пользователь заполняет форму, данные сохраняются в базе
52                 данных и доступны через административную панель Django.</p>
53             </details>
54             <details>
55                 <summary>Можно ли подключить email-уведомления?</summary>
56                 <p>Да, используя встроенные инструменты Django
57                 (например, EmailBackend), можно настроить отправку писем.</p>
58             </details>
59         </section>
60     </div>
61 {% endblock %}

```

Рисунок 13 — Код файла *about.html*.

Последняя страница – страница обратной связи *contact.html*.

```
1  {% extends 'main/base.html' %}
2  {% block title %}Обратная связь{% endblock %}
3  {% block content %}
4      <h2>Форма обратной связи</h2>
5      {% if messages %}
6          {% for message in messages %}
7              <div class="message {{ message.tags }}">{{ message }}</div>
8          {% endfor %}
9      {% endif %}
10
11     <form method="post" class="contact-form">
12         {% csrf_token %}
13         <div class="form-group">
14             <label for="id_name">Ваше имя</label>
15             {{ form.name }}
16         </div>
17         <div class="form-group">
18             <label for="id_email">Ваш email</label>
19             {{ form.email }}
20         </div>
21         <div class="form-group">
22             <label for="id_subject">Тема</label>
23             {{ form.subject }}
24         </div>
25         <div class="form-group">
26             <label for="id_message">Сообщение</label>
27             {{ form.message }}
28         </div>
29         <button type="submit" class="btn-submit">Отправить</button>
30     </form>
31 {% endblock %}
```

Рисунок 14 — Код файла *about.html*.

Стили для всех файлов лежат в *styles.css*. В итоге получаем следующую структуру для *static*.

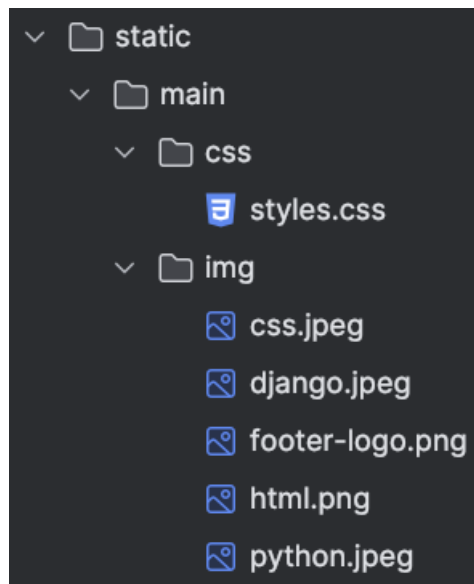


Рисунок 15 — Структура каталога *static*.

Создадим модель обратной связи.

```
1  from django.db import models
2
3  class ContactMessage(models.Model): 2 usages
4      name = models.CharField(max_length=100)
5      email = models.EmailField()
6      message = models.TextField()
7      subject = models.CharField(max_length=255, default='No subject')
8      submitted_at = models.DateTimeField(auto_now_add=True)
9
10  def __str__(self):
11      return self.subject
```

Рисунок 16 — Код файла *main/models.py*.

Так как мы хотим просматривать сообщения через Django Admin, добавим код в *main/admin.py*

```

1  from django.contrib import admin
2  from .models import ContactMessage
3
4  @admin.register(ContactMessage)
5  class ContactMessageAdmin(admin.ModelAdmin):
6      # Добавляем 'subject' в list_display
7      list_display = ('name', 'email', 'message', 'subject', 'submitted_at')
8
9      # Добавляем 'subject' в readonly_fields
10     readonly_fields = ('name', 'email', 'message', 'subject', 'submitted_at')
11
12     # Устанавливаем порядок отображения
13     ordering = ('-submitted_at',)

```

Рисунок 17 — Код файла *main/admin.py*.

В конце создадим superuser'а для входа в Django Admin.

```

(.venv) aleksandrbacanov@MacBook-Pro-Aleksandr-3 mysite % python manage.py createsuperuser

Username (leave blank to use 'aleksandrbacanov'): kathy
Email address: kathy.a.b.2004@gmail.com
Password:
Password (again):
This password is too short. It must contain at least 8 characters.
This password is too common.
This password is entirely numeric.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.

```

Рисунок 18 — Создание superuser'а.

Для того чтобы снова запустить сервер необходимо создать файлы миграций на основе изменений в моделях (*models.py*) с помощью

```
python manage.py makemigrations
```

И после этого как раньше применить миграции.


```
(.venv) aleksandrbaranov@MacBook-Pro-Aleksandr-3 mysite % python manage.py makemigrations
Migrations for 'main':
  main/migrations/0001_initial.py
    + Create model ContactMessage
(.venv) aleksandrbaranov@MacBook-Pro-Aleksandr-3 mysite % python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, main, sessions
Running migrations:
  Applying main.0001_initial... OK
```

Рисунок 19 — Применение миграций.

Запустим сервер и посмотрим на наш сайт.

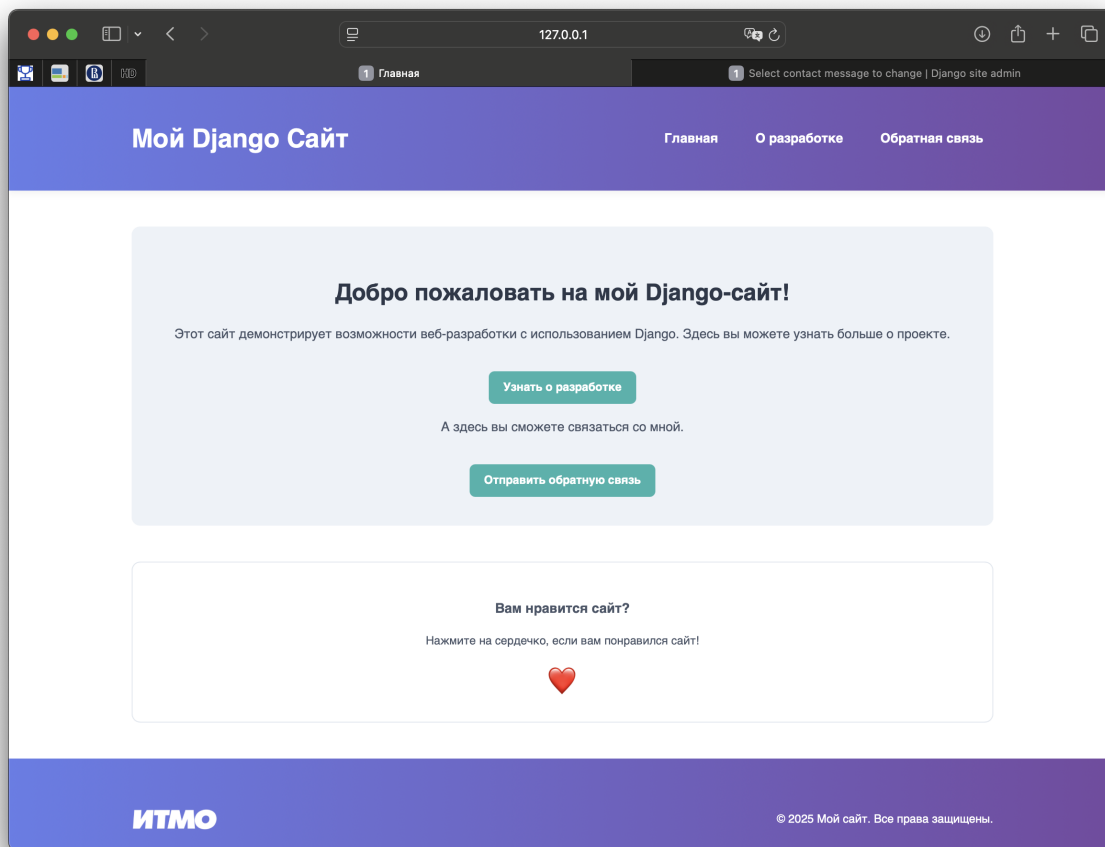


Рисунок 20 — Вид главной страницы в браузере.

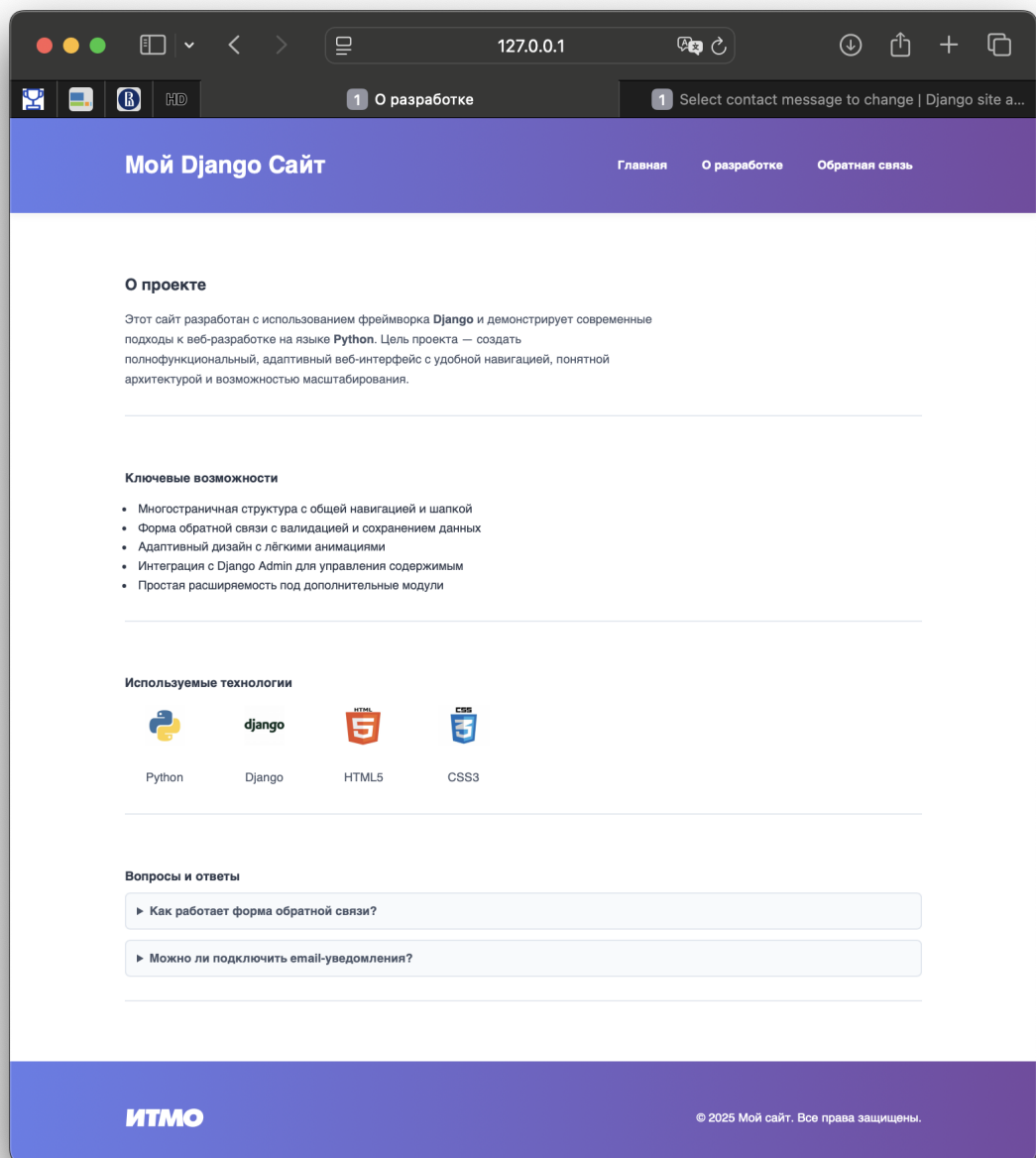


Рисунок 21 — Вид страницы о разработке в браузере.

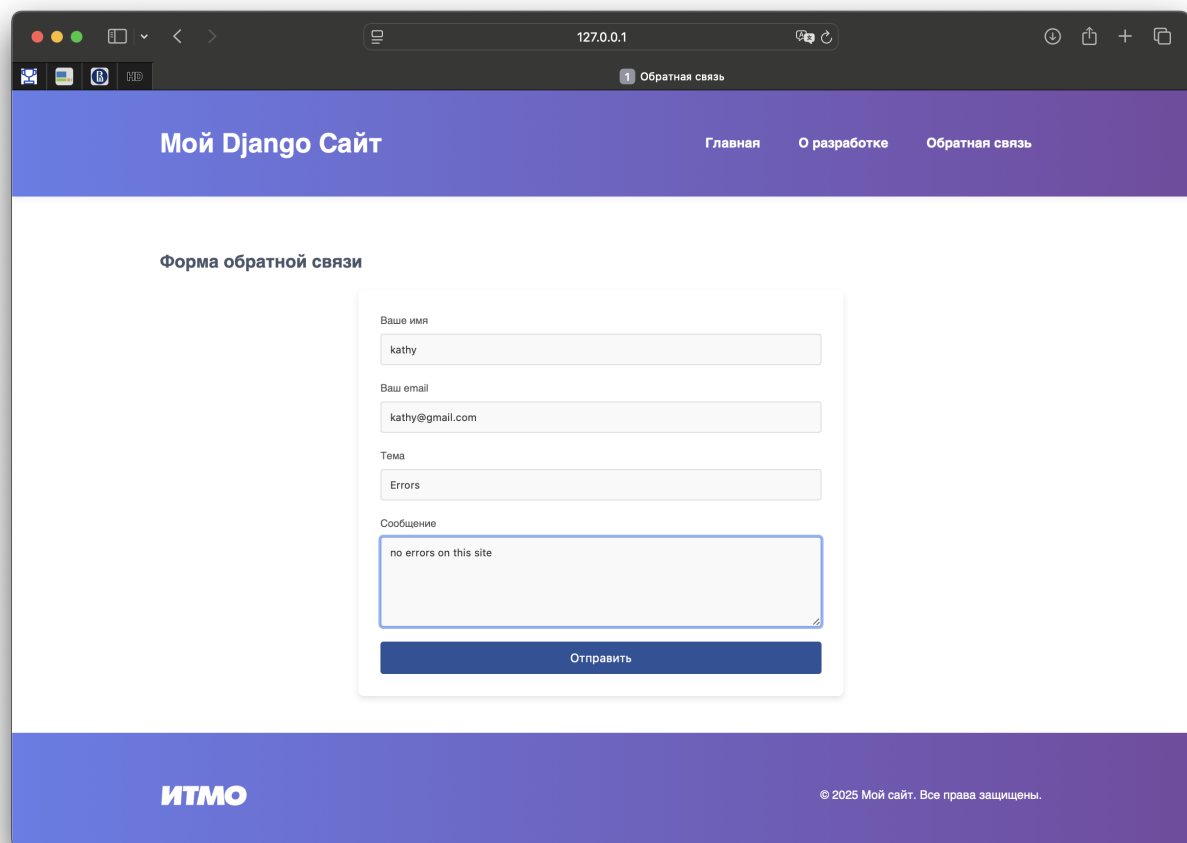


Рисунок 22 — Вид страницы обратной связи в браузере.

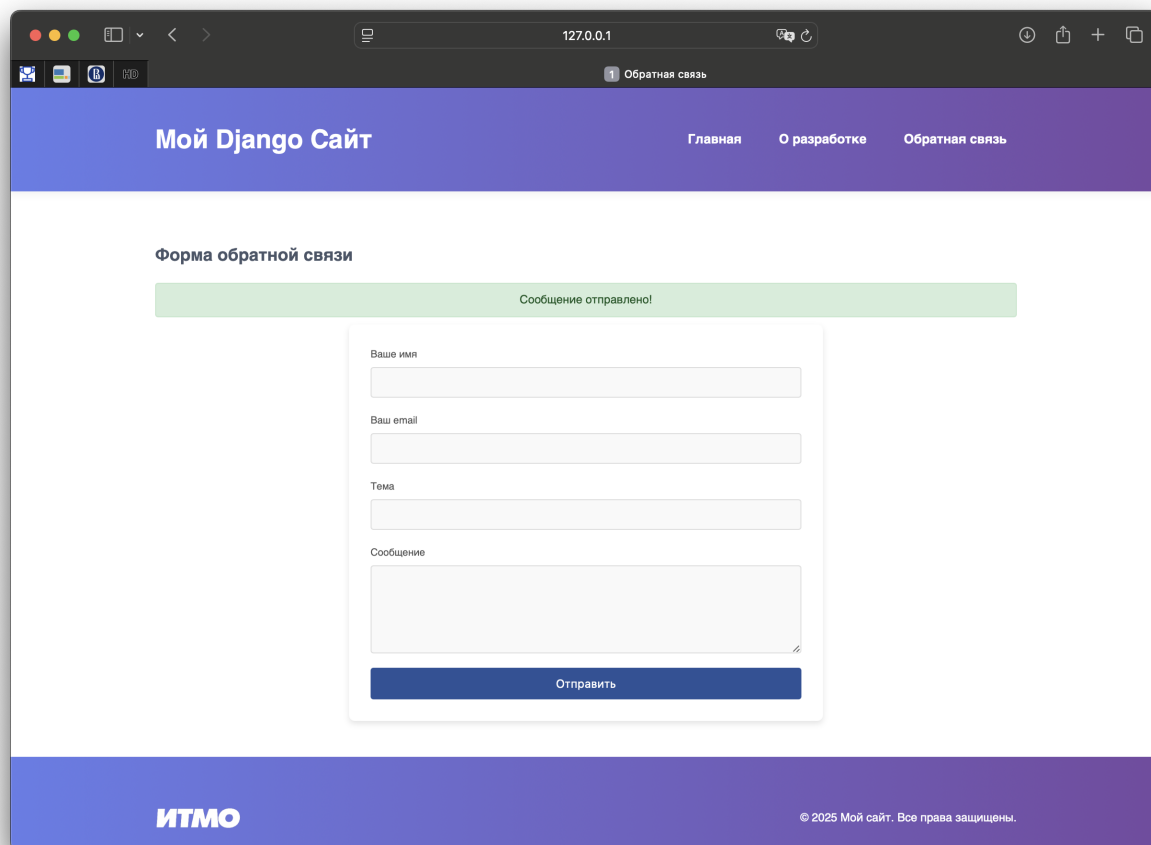


Рисунок 23 — Вид страницы обратной связи в браузере после отправки сообщения.

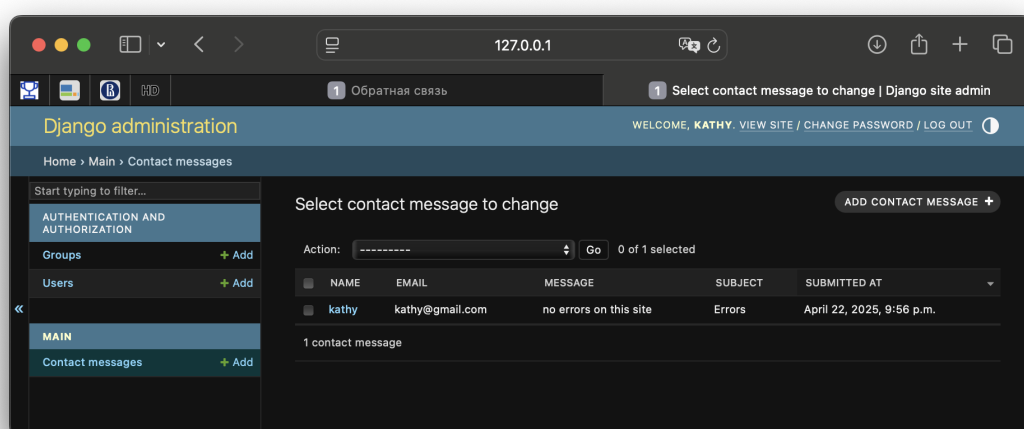


Рисунок 24 — Сохраненное сообщение в Django Admin.

ВЫВОД

В ходе работы был изучен готовый проект на Django. Он помог понять структуру фреймворка, принципы маршрутизации, подключения шаблонов и обработки форм.

Затем был создан собственный проект с нуля, включающий три страницы: главную, о разработке и страницу обратной связи. Для всех страниц использовался общий шаблон с единым хедером и футером. Основной акцент сделан на реализации формы обратной связи: с помощью встроенных возможностей Django создана и подключена форма, реализована её проверка и вывод уведомления об успешной отправке.

Проект демонстрирует применение шаблонов, представлений, маршрутов и форм, а также позволяет закрепить навыки работы с Django на практике.